

# ANTI-PATTERNS - PARTE 2: CENÁRIOS FINAIS E ATIVIDADE PRÁTICA

## CENÁRIO 4: ACOPLAMENTO CONCRETO (Tight Coupling Anti-Pattern)

### O Código Problemático

Este anti-pattern representa um dos erros mais fundamentais que desenvolvedores cometem quando estão começando a aprender sobre factories: criar uma factory mas ainda assim manter acoplamento forte com implementações concretas. O código funciona, mas perde a maioria dos benefícios que factories deveriam fornecer.

csharp

```
using System;
```

```
// Factory que está acoplada a implementações concretas
```

```
public class NotificationFactory
```

```
{
```

```
    // Configuração hardcoded - não vem de config externa
```

```
    private const string SmtpServer = "smtp.gmail.com";
```

```
    private const int SmtpPort = 587;
```

```
    private const string SmsProvider = "twilio";
```

```
    private const string SmsApiKey = "hardcoded-api-key-12345";
```

```
    public INotifier CreateEmailNotifier(string recipient)
```

```
    {
```

```
        // Instanciação direta com parâmetros hardcoded
```

```
        var notifier = new EmailNotifier();
```

```
        notifier.SmtpServer = SmtpServer;
```

```
        notifier.Port = SmtpPort;
```

```
        notifier.Recipient = recipient;
```

```
        // Criando dependências internas diretamente
```

```
        notifier.SmtpClient = new SmtpClient(SmtpServer, SmtpPort);
```

```
        notifier.Logger = new FileLogger("/var/log/email.log");
```

```
        return notifier;
```

```
    }
```

```
    public INotifier CreateSmsNotifier(string phoneNumber)
```

```
    {
```

```
        // Mais instanciação direta
```

```
        var notifier = new SmsNotifier();
```

```
        notifier.Provider = SmsProvider;
```

```
        notifier.ApiKey = SmsApiKey;
```

```
        notifier.PhoneNumber = phoneNumber;
```

```
        // Criando client HTTP diretamente
```

```
        notifier.HttpClient = new System.Net.Http.HttpClient();
```

```
        notifier.HttpClient.BaseAddress = new Uri("https://api.twilio.com");
```

```
        notifier.Logger = new FileLogger("/var/log/sms.log");
```

```
        return notifier;
```

```
    }
```

```
    public INotifier CreatePushNotifier(string deviceToken)
```

```
    {
```

```
        // Ainda mais acoplamento
```

```
        var notifier = new PushNotifier();
```

```
notifier.DeviceToken = deviceToken;
```

```
// Criando dependências específicas
```

```
notifier.FirebaseClient = new FirebaseClient("hardcoded-firebase-key");
```

```
notifier.Logger = new FileLogger("/var/log/push.log");
```

```
return notifier;
```

```
}
```

```
// Implementações concretas com muitas dependências
```

```
public class EmailNotifier : INotifier
```

```
{
```

```
    public string SmtpServer { get; set; }
```

```
    public int Port { get; set; }
```

```
    public string Recipient { get; set; }
```

```
    public SmtpClient SmtpClient { get; set; } // Dependência concreta
```

```
    public FileLogger Logger { get; set; } // Dependência concreta
```

```
    public void Send(string message)
```

```
    {
```

```
        Logger.Log($"Enviando email para {Recipient}");
```

```
        SmtpClient.Send(Recipient, message);
```

```
    }
```

```
}
```

```
public class SmsNotifier : INotifier
```

```
{
```

```
    public string Provider { get; set; }
```

```
    public string ApiKey { get; set; }
```

```
    public string PhoneNumber { get; set; }
```

```
    public System.Net.Http.HttpClient HttpClient { get; set; }
```

```
    public FileLogger Logger { get; set; }
```

```
    public void Send(string message)
```

```
    {
```

```
        Logger.Log($"Enviando SMS para {PhoneNumber}");
```

```
        // Usar HttpClient para chamar API SMS
```

```
    }
```

```
}
```

```
public class PushNotifier : INotifier
```

```
{
```

```
    public string DeviceToken { get; set; }
```

```
    public FirebaseClient FirebaseClient { get; set; }
```

```
    public FileLogger Logger { get; set; }
```

```
public void Send(string message)
{
    Logger.Log($"Enviando push para {DeviceToken}");
    FirebaseClient.SendPush(DeviceToken, message);
}
}

// Dependências concretas problemáticas
public class SmtplibClient
{
    public SmtplibClient(string server, int port) { }
    public void Send(string recipient, string message)
    {
        Console.WriteLine($"[SMTP] Enviando para {recipient}: {message}");
    }
}

public class FileLogger
{
    private readonly string _logPath;

    public FileLogger(string logPath)
    {
        _logPath = logPath;
    }

    public void Log(string message)
    {
        Console.WriteLine($"[LOG to {_logPath}] {message}");
        // Em produção, escreveria para arquivo
    }
}

public class FirebaseClient
{
    public FirebaseClient(string apiKey) { }
    public void SendPush(string deviceToken, string message)
    {
        Console.WriteLine($"[Firebase] Enviando push para {deviceToken}: {message}");
    }
}

public interface INotifier
{
    void Send(string message);
}
```

## Análise dos Problemas

Este código ilustra vários níveis de acoplamento forte que trabalham juntos para tornar o sistema rígido e difícil de modificar. O primeiro problema é configuração hardcoded. Valores como servidor SMTP, chaves de API e endereços estão codificados diretamente no factory. Se você precisar mudar para um servidor SMTP diferente ou usar um provedor SMS diferente, você precisa modificar o código fonte e recompilar.

O segundo problema é instanciação direta de dependências. A factory cria `SmtplibClient`, `HttpClient`, `FirebaseClient` e `FileLogger` usando `new`. Isso cria acoplamento direto com estas implementações concretas específicas. Se você quisesse substituir `FileLogger` por `DatabaseLogger`, você precisaria modificar o factory.

O terceiro problema é que os notifiers têm propriedades públicas para suas dependências (`SmtplibClient`, `Logger`, etc.), que é um code smell. Dependências deveriam ser injetadas via construtor e mantidas privadas. Propriedades públicas permitem que código externo modifique as dependências depois que o objeto foi criado, violando encapsulamento.

O quarto problema é falta de abstrações para dependências. `SmtplibClient`, `HttpClient` e `FirebaseClient` são tipos concretos, não interfaces. Isto torna impossível substituí-los por mocks em testes ou por implementações alternativas em produção.

## A Refatoração Completa

csharp

*// PASSO 1: Criar abstrações para todas as dependências*

```
public interface IEmailSender
{
    void SendEmail(string recipient, string message);
}

public interface ISmsSender
{
    void SendSms(string phoneNumber, string message);
}

public interface IPushSender
{
    void SendPush(string deviceToken, string message);
}

public interface ILogger
{
    void Log(string message);
}
```

*// PASSO 2: Refatorar notifiers para usar constructor injection*

```
public class EmailNotifier : INotifier
{
    private readonly string _recipient;
    private readonly IEmailSender _emailSender;
    private readonly ILogger _logger;

    // Constructor injection de todas as dependências
    public EmailNotifier(
        string recipient,
        IEmailSender emailSender,
        ILogger logger)
    {
        _recipient = recipient ?? throw new ArgumentNullException(nameof(recipient));
        _emailSender = emailSender ?? throw new ArgumentNullException(nameof(emailSender));
        _logger = logger ?? throw new ArgumentNullException(nameof(logger));
    }

    public void Send(string message)
    {
        _logger.Log($"Enviando email para {_recipient}");
        _emailSender.SendEmail(_recipient, message);
    }
}
```

```

    }

    public class SmsNotifier : INotifier
    {
        private readonly string _phoneNumber;
        private readonly ISmsSender _smsSender;
        private readonly ILogger _logger;

        public SmsNotifier(
            string phoneNumber,
            ISmsSender smsSender,
            ILogger logger)
        {
            _phoneNumber = phoneNumber ?? throw new ArgumentNullException(nameof(phoneNumber));
            _smsSender = smsSender ?? throw new ArgumentNullException(nameof(smsSender));
            _logger = logger ?? throw new ArgumentNullException(nameof(logger));
        }

        public void Send(string message)
        {
            _logger.Log($"Enviando SMS para {_phoneNumber}");
            _smsSender.SendSms(_phoneNumber, message);
        }
    }
}

```

*// PASSO 3: Refatorar factory para usar DI*

```

public class NotificationFactory : INotificationFactory
{
    private readonly IServiceProvider _serviceProvider;
    private readonly ILogger _logger;

    public NotificationFactory(
        IServiceProvider serviceProvider,
        ILogger logger)
    {
        _serviceProvider = serviceProvider;
        _logger = logger;
    }

    public INotifier CreateEmailNotifier(string recipient)
    {
        // Resolver dependências do container
        var emailSender = _serviceProvider.GetRequiredService<IEmailSender>();
        var logger = _serviceProvider.GetRequiredService<ILogger>();

        // Criar notifier com dependências resolvidas
    }
}

```

```

        return new EmailNotifier(recipient, emailSender, logger);
    }

    public INotifier CreateSmsNotifier(string phoneNumber)
    {
        var smsSender = _serviceProvider.GetService<ISmsSender>();
        var logger = _serviceProvider.GetService<ILogger>();
        return new SmsNotifier(phoneNumber, smsSender, logger);
    }
}

// PASSO 4: Implementações concretas das abstrações

public class SmtpEmailSender : IEmailSender
{
    private readonly string _smtpServer;
    private readonly int _port;

    // Configuração vem de IConfiguration
    public SmtpEmailSender(IConfiguration configuration)
    {
        _smtpServer = configuration["Email:SmtpServer"];
        _port = int.Parse(configuration["Email:Port"]);
    }

    public void SendEmail(string recipient, string message)
    {
        Console.WriteLine($"[SMTP { _smtpServer}] Enviando para {recipient}: {message}");
    }
}

public class TwilioSmsSender : ISmsSender
{
    private readonly string _apiKey;

    public TwilioSmsSender(IConfiguration configuration)
    {
        _apiKey = configuration["Sms:ApiKey"];
    }

    public void SendSms(string phoneNumber, string message)
    {
        Console.WriteLine($"[Twilio] SMS para {phoneNumber}: {message}");
    }
}

```

*// PASSO 5: Registrar tudo no DI container*



```
public void ConfigureServices(IServiceCollection services, IConfiguration configuration)
{
    // Registrar implementações das abstrações
    services.AddSingleton<IEmailSender, SmtplibEmailSender>();
    services.AddSingleton<ISmsSender, TwilioSmsSender>();
    services.AddSingleton<ILogger, ConsoleLogger>(); // ou FileLogger, DatabaseLogger, etc.

    // Registrar factory
    services.AddSingleton<INotificationFactory, NotificationFactory>();
}
```

## Benefícios Alcançados

Agora o código é fracamente acoplado, configurável e testável. Configuração vem de fontes externas. Dependências são injetadas via construtor. Tudo depende de abstrações. Você pode substituir qualquer implementação sem modificar o factory.

---

## CENÁRIO 5: DEPENDÊNCIAS CATIVAS (Captive Dependencies Anti-Pattern)

### O Problema Sutil Mas Sério

Captive Dependencies é um anti-pattern relacionado a lifetimes no DI container que causa bugs sutis relacionados a estado compartilhado, memory leaks e comportamento inesperado. O problema ocorre quando um serviço de lifetime longo (Singleton) captura uma dependência de lifetime curto (Scoped ou Transient).

csharp

```
using Microsoft.Extensions.DependencyInjection;
using Microsoft.EntityFrameworkCore;
```

*// PROBLEMA: Singleton captura dependência Scoped*

*// DbContext é Scoped (uma instância por requisição)*

```
public class AppDbContext : DbContext
{
    public DbSet<Order> Orders { get; set; }

    public AppDbContext(DbContextOptions<AppDbContext> options)
        : base(options)
    {
    }
}
```

*// Repository injeta DbContext*

```
public class OrderRepository
{
    private readonly AppDbContext _context;

    public OrderRepository(AppDbContext context)
    {
        _context = context;
    }

    public void SaveOrder(Order order)
    {
        _context.Orders.Add(order);
        _context.SaveChanges();
    }
}
```

*// Factory é Singleton mas captura OrderRepository que depende de DbContext Scoped*

```
public class OrderFactory
{
    // PROBLEMA: Este campo captura uma dependência Scoped
    private readonly OrderRepository _repository;

    public OrderFactory(OrderRepository repository)
    {
        // Repository injetado aqui será o mesmo para toda vida da aplicação
        // Mas repository depende de DbContext que deveria ser Scoped!
        _repository = repository;
    }
}
```

```

public Order CreateOrder(string customerId)
{
    var order = new Order
    {
        Id = Guid.NewGuid(),
        CustomerId = customerId,
        CreatedAt = DateTime.Now
    };

    // Usando repository capturado - PERIGO!
    _repository.SaveOrder(order);

    return order;
}

// Registro problemático
public void ConfigureServices(IServiceCollection services)
{
    services.AddDbContext<AppDbContext>(options =>
        options.UseInMemoryDatabase("TestDb")); // DbContext é Scoped por padrão

    services.AddScoped<OrderRepository>(); // Scoped

    services.AddSingleton<OrderFactory>(); // Singleton captura Scoped - PROBLEMA!
}

public class Order
{
    public Guid Id { get; set; }
    public string CustomerId { get; set; }
    public DateTime CreatedAt { get; set; }
}

```

## Por Que É Problemático

O problema é que OrderFactory é Singleton (uma única instância para toda aplicação), mas ela injeta OrderRepository que é Scoped. Quando o Singleton é criado no startup, ele captura uma instância do OrderRepository. Este OrderRepository específico fica "cativo" dentro do Singleton e será reutilizado para todas as requisições durante toda a vida da aplicação.

Isto causa múltiplos problemas sérios. Primeiro, o DbContext que está dentro do OrderRepository capturado também fica preso no Singleton. DbContext foi desenhado para ser Scoped - uma nova instância por requisição. Mas agora você tem um único DbContext compartilhado por todas as requisições, o que viola completamente o modelo de uso do Entity Framework Core e pode causar exceções, corrupção de dados e race conditions.

Segundo, memória não é liberada corretamente. Objetos Scoped normalmente seriam descartados (disposed) ao fim de cada scope (requisição). Mas quando capturados por um Singleton, eles nunca são descartados, causando memory leaks.

Terceiro, estado pode vazar entre requisições. Se o OrderRepository mantém qualquer estado, esse estado é compartilhado entre todas as requisições, causando bugs onde dados de um usuário vazam para outro usuário.

## A Solução Correta

```
csharp
```

*// SOLUÇÃO 1: Factory resolve dependências just-in-time*

```
public class OrderFactory
{
    // Inject IServiceProvider em factory (aceitável aqui)
    private readonly IServiceProvider _serviceProvider;

    public OrderFactory(IServiceProvider serviceProvider)
    {
        _serviceProvider = serviceProvider;
    }

    public Order CreateOrder(string customerId)
    {
        // Criar um scope explicitamente
        using (var scope = _serviceProvider.CreateScope())
        {
            // Resolver repository dentro do scope
            var repository = scope.ServiceProvider.GetRequiredService<OrderRepository>();

            var order = new Order
            {
                Id = Guid.NewGuid(),
                CustomerId = customerId,
                CreatedAt = DateTime.Now
            };

            repository.SaveOrder(order);

            return order;
        } // Scope é descartado, liberando DbContext
    }
}
```

*// SOLUÇÃO 2: Usar Func<T> para lazy resolution*

```
public class OrderFactory
{
    // Inject factory function
    private readonly Func<OrderRepository> _repositoryFactory;

    public OrderFactory(Func<OrderRepository> repositoryFactory)
    {
        _repositoryFactory = repositoryFactory;
    }
}
```

```

public Order CreateOrder(string customerId)
{
    // Resolver repository just-in-time
    var repository = _repositoryFactory();

    var order = new Order
    {
        Id = Guid.NewGuid(),
        CustomerId = customerId,
        CreatedAt = DateTime.Now
    };

    repository.SaveOrder(order);

    return order;
}

// Registro para Solução 2
services.AddDbContext<AppDbContext>();
services.AddScoped<OrderRepository>();
services.AddSingleton<Func<OrderRepository>>(provider =>
{
    return () =>
    {
        // Criar novo scope e resolver
        var scope = provider.CreateScope();
        return scope.ServiceProvider.GetRequiredService<OrderRepository>();
    };
});
services.AddSingleton<OrderFactory>();

// SOLUÇÃO 3: Mudar lifetime do factory (mais simples se possível)

// Se factory não precisa ser Singleton, mude para Scoped
services.AddScoped<OrderFactory>(); // Agora pode injetar OrderRepository diretamente

```

## Como Detectar Captive Dependencies

Use ferramentas de análise estática como o pacote NuGet "Scrutor" ou "Microsoft.Extensions.DependencyInjection.Analyzers" que detectam estes problemas em tempo de compilação. Em runtime, preste atenção a exceções relacionadas a DbContext sendo acessado depois de disposed, ou comportamento onde estado vaza entre requisições diferentes.

---

# ATIVIDADE PRÁTICA: CODE SMELL DETECTIVE

## Descrição da Atividade

Esta atividade foi projetada para desenvolver sua habilidade de reconhecer anti-patterns em código real. Você receberá três projetos pequenos mas completos, cada um contendo múltiplos anti-patterns estrategicamente colocados. Seu objetivo é identificar todos os problemas, explicar por que são problemáticos, e propor refatorações.

## Projeto 1: Sistema de Processamento de Pedidos

Baixe o código completo deste projeto de exemplo (simulado aqui):

csharp

// OrderProcessor.cs - ENCONTRE OS PROBLEMAS!

```
public class OrderProcessor
{
    private static int _orderCount = 0;
    private static Dictionary<string, decimal> _orderTotals = new Dictionary<string, decimal>();

    public Order ProcessOrder(Customer customer, List<Product> products)
    {
        _orderCount++;

        var order = new Order
        {
            Id = _orderCount.ToString(),
            CustomerId = customer.Id,
            Items = products
        };

        // Calcular total
        decimal total = 0;
        foreach (var product in products)
        {
            var pricing = new PricingService();
            var price = pricing.GetPrice(product.Id);
            total += price;
        }

        order.Total = total;
        _orderTotals[customer.Id] = total;

        // Salvar pedido
        var connection = new SqlConnection("Server=localhost;Database=Store");
        connection.Open();
        var command = new SqlCommand($"INSERT INTO Orders VALUES ({order.Id}, '{customer.Id}', {total})", connection);
        command.ExecuteNonQuery();
        connection.Close();

        // Enviar notificação
        var smtp = new SmtpClient("smtp.company.com");
        smtp.Send("noreply@company.com", customer.Email, "Pedido Confirmado", $"Seu pedido #{order.Id} foi confirmado.");

        return order;
    }

    public static Dictionary<string, decimal> GetAllOrderTotals()
    {
        return _orderTotals;
    }
}
```



```
}  
}  
  
public class PricingService  
{  
    public decimal GetPrice(string productId)  
    {  
        // Simular busca de preço  
        return 99.90m;  
    }  
}  
  
// Outros arquivos do projeto...
```

## Tarefas Para Este Projeto

Tarefa um: Identificar todos os anti-patterns presentes. Liste cada problema que você encontrar. Para cada problema, dê um nome (pode ser um dos anti-patterns discutidos ou um novo code smell que você identifica).

Tarefa dois: Para cada problema identificado, explique em três a cinco frases por que é problemático. Considere testabilidade, manutenibilidade, corretude, performance e princípios SOLID.

Tarefa três: Propor uma refatoração específica para cada problema. Mostre código antes e depois. Explique como a refatoração resolve o problema sem introduzir novos problemas.

Tarefa quatro: Escrever pelo menos três testes unitários que demonstrem os problemas do código original e mostram que sua refatoração os resolve.

## Dicas de Análise

Ao analisar código procurando por anti-patterns, siga este checklist mental. Primeiro, verifique estado estático ou compartilhado. Qualquer campo static mutável é suspeito. Em código multi-threaded, estado compartilhado mutável causa race conditions.

Segundo, procure instanciação direta com new. Cada vez que você vê new seguido de uma classe concreta, pergunte se isso cria acoplamento forte. Poderia ser injetado ao invés disso?

Terceiro, identifique strings mágicas e valores hardcoded. Connection strings, URLs, chaves de API hardcodedas no código são problemas. Configuração deve vir de fontes externas.

Quarto, analise responsabilidades da classe. Quantas coisas diferentes esta classe faz? Se mais de uma responsabilidade clara, viola SRP.

Quinto, examine testabilidade. Imagine tentar testar esta classe. Quão difícil seria? Se muito difícil, provavelmente há problemas de design.

## Entrega da Atividade

Prepare um relatório contendo para cada projeto analisado a lista numerada de todos os anti-patterns encontrados, explicação detalhada de por que cada um é problemático, código refatorado demonstrando correção dos problemas, testes unitários provando que refatoração mantém comportamento correto enquanto resolve problemas, e reflexão pessoal sobre o que você aprendeu identificando e corrigindo estes problemas.

O relatório deve ser entregue como documento Markdown ou PDF, com código em formato legível e bem formatado. Inclua diagramas se ajudarem a explicar problemas ou soluções.

---

## CONCLUSÃO E PRÓXIMOS PASSOS

### O Que Você Aprendeu

Através deste material de anti-patterns, você desenvolveu uma habilidade crítica: reconhecer code smells e problemas de design em código que aparentemente funciona. Esta habilidade o distingue como desenvolvedor porque você agora pode antecipar problemas antes que eles se manifestem em produção.

Você aprendeu que God Factory viola responsabilidade única e cria gargalos de manutenção. Service Locator esconde dependências e compromete testabilidade. Stateful Factory causa race conditions e memory leaks. Tight Coupling torna código rígido e difícil de modificar. Captive Dependencies criam bugs sutis relacionados a lifetimes.

Mais importante, você aprendeu processos mentais de análise. Você aprendeu a questionar estado compartilhado, a identificar acoplamento forte, a avaliar responsabilidades de classes, a considerar testabilidade como indicador de qualidade de design.

### Aplicando Este Conhecimento

Use este conhecimento ativamente em code reviews. Quando revisar código de colegas, procure estes anti-patterns. Mas seja construtivo em feedback - explique por que algo é problemático e proponha soluções alternativas.

Aplique em seu próprio código. Antes de fazer commit, revise seu próprio código procurando por estes anti-patterns. Pergunte-se: este código será fácil de testar? Se eu precisar mudar isso no futuro, quão difícil será? Estou criando acoplamento desnecessário?

Refatore código legacy. Quando trabalhar em código existente que contém anti-patterns, refatore incrementalmente. Não tente refatorar tudo de uma vez. Escolha um anti-pattern, corrija-o, teste, commit. Depois passe para o próximo.

### Leituras Complementares

Para aprofundar conhecimento sobre anti-patterns e design patterns em geral, recomendo "Refactoring" de Martin Fowler que ensina a identificar code smells e aplicar refatorações sistemáticas. "Working Effectively

with Legacy Code" de Michael Feathers que ensina técnicas específicas para lidar com código problemático existente. "Clean Code" de Robert Martin que estabelece princípios fundamentais de código limpo.

### **Materiais de Referência Rápida**

Mantenha um checklist de anti-patterns próximo enquanto desenvolve. Quando você terminar de implementar uma factory, percorra o checklist verificando que não introduziu nenhum destes problemas. Com o tempo, estes checks se tornarão automáticos.

---

### **FIM DO GUIA DE ANTI-PATTERNS**

Este material complementa o curso principal de Factory Patterns e DI, fornecendo a perspectiva crítica necessária para não apenas implementar padrões corretamente mas também evitar armadilhas comuns e reconhecer quando código precisa ser refatorado.